

408 真题·虚拟内存知识点梳理

由 2009--2026 共 65 道真题的解析归纳

每条结论后标出出处（年份-题号）；标 **✗** 的都是真题里被设成错误选项的说法

一、地址结构与位数拆分

这是虚拟内存题的**第一步**。核心思路：地址的每一位都有归属，**先定页内偏移，剩下的全归页号**。

量	求法
页内偏移位数	k ，其中页大小 = 2^k
虚页号位数	虚拟地址位数 $-k$
页框号位数	物理地址位数 $-k$
虚拟地址位数	$\log_2(\text{虚拟地址空间大小})$
物理地址位数	$\log_2(\text{主存地址空间大小})$
页表项数	$2^{\text{虚页号位数}}$
一页能放的页表项数	页大小 $\div$ 页表项大小

物理地址 = 页框号拼接页内偏移，不是相加。因为页大小是 2 的幂，拼接恰好等价于「页框号 $\times$ 页大小 + 页内偏移」。

出处：2011-44、2013-16、2013-46、2015-46、2019-31、2020-46、2021-44、2022-15、2024-17、2024-45、2026-19

关键推论：页大小 = $2^k \implies$ 虚拟地址与物理地址的**低 k 位完全相同**。这是第九节「与 Cache 交叉」要用的结论。

单位易错：2022-15 的页表里虚页号那列末尾没写 H，是**十进制**，查表前必须先转换。

二、多级页表

经典结构：二级页表对应 32 位虚拟地址、4KB 页、4B 页表项时，地址拆成「页目录号 (10) | 页号 (10) | 页内偏移 (12)」。

唯一优点：减少页表占用的**连续**内存空间。把一张大页表分级，每张控制在一页之内。

它**不**加快地址变换（反而多一次查表、更慢）、**不**减少缺页中断（多级页表都不在内存时反而更多）、**不**减少页表项所占字节数。2014-32 的四个选项就是照这个设的。

出处：2010-29、2014-32、2015-46、2019-31、2020-46、2021-29、2026-28

页目录表项数 (2010-29)：逻辑地址空间 2^{16} 页，每页放 $2^{10}/2 = 2^9$ 个页表项，故 $2^{16}/2^9 = 2^7 = 128$ 。

页目录与页表各占多少页 (2015-46)：页目录 = $2^{10} \times 4B / 4KB = 1$ 页；页表 = $2^{20} \times 4B / 4KB = 1024$ 页。

页表项物理地址 (2013-46、2020-46、2024-45)：

$$\text{页表项物理地址} = \text{页表起始物理地址} + \text{页号} \times \text{页表项大小}$$

页表基址寄存器 (2021-29、2018-45)：PTBR / PDBR 里存的是当前进程**一级页表的起始物理地址**。页表本身不参与虚拟地址的变换，CPU 是直接拿这个物理地址去访问页表的。

三级页表页框数 (2026-28)：每级索引 n 位 $\implies$ 每张页表 2^n 项；最末级页表的张数 = 各级索引容量的乘积。

三、TLB

本质：TLB 就是「页表的 Cache」，所以它的索引方式和 Cache 完全一样，按处理 Cache 的方式求解即可。

量	求法
组数	表项数 $\div$ 路数
组号位数	$\log_2(\text{组数})$
TLB 标记位数	虚页号位数 - 组号位数
组号映射	TLB 组号 = 虚页号 mod 组数

TLB 标记字段里放的是虚页号的高位部分（2016-45 明确问过）。

命中关系铁律（2010-17）：

- Cache 命中 $\implies$ Page 必命中（数据既然在主存，其所在页必然已调入）
- TLB 命中 $\implies$ Page 必命中
- TLB 未命中 $\implies$ Page **可能**命中（快表只是页表的部分副本，该表项可能没被缓存）
- **不可能发生：**TLB 命中 + Page 未命中

用什么实现（2020-15）：TLB 用相联存储器（也可用 SRAM），Cache 用 SRAM。**都不能用 DRAM**——需要不断刷新，太慢。

检测时机（2024-18）：MMU 在地址转换过程中检测「访问越权、页面缺失、TLB 缺失」三件事。**Cache 缺失不在其中**——它发生在地址翻译完成之后，CPU 拿物理地址访问主存时才检测。

加快虚实地址转换（2014-28）：增大 TLB 容量 $\checkmark$ 、让页表常驻内存 $\checkmark$ 、增大交换区 $\times$ （与转换速度无关）。

四、缺页异常

标准处理流程（2011-28、2013-30）：

1. 在内存中找空闲页框并分配（可能涉及页面置换）
2. 调用设备驱动程序做磁盘 I/O，把缺失的页调入内存
3. 修改页表：存在位（有效位）置 1，填入物理页框号

不一定包含「淘汰页」（2022-29）——内存里还有空闲页框时直接分配就行，不必淘汰。

返回位置（2019-14）：缺页处理完成后，返回到**发生缺页的那条指令**重新执行，**不是下一条**。这是最经典的错误选项。

只有访存指令会缺页（2014-45）——加法、移位、寄存器自增这类运算指令都不会。

一次缺页 = 多访问一次 TLB（2014-45）：第一次访问某页要查一次 TLB，触发缺页；处理完重新执行这条指令，又要查一次 TLB。所以访问 n 次数据、其中一次缺页，TLB 访问次数是 $n + 1$ 。

哪个组件做什么（2026-24）：硬件（MMU）做地址转换、硬件触发缺页中断；**操作系统**管理页表、设置页表项内容、处理异常。页表项内容**不是编译器**确定的。

五、页面置换算法

Belady 异常 (2014-30): 只有 **FIFO** 会产生——页框数增加, 缺页次数反而增加。LRU 和 OPT 都不会。

LRU 手算技巧 (2015-27): 从访问序列**从后往前**数, 数到第 n 个 ($n =$ 页框数) **不同的**页号, 停下时那个页号就是被淘汰的。

改进型 CLOCK 淘汰次序 (2016-26): 按 (访问位 A, 修改位 M) 把页分四类, 淘汰顺序为

$$(0, 0) \rightarrow (0, 1) \rightarrow (1, 0) \rightarrow (1, 1)$$

先找最近没访问过的; 同是没访问过, 优先淘汰没修改过的 (不必写回磁盘, 代价小)。

基本 CLOCK (2010-46): 指针扫过页框, 使用位为 1 就清零并继续, 为 0 就替换该页; 新页装入后把使用位置 1。

置换次数 $\neq$ 缺页次数 (2019-29) ——进程刚启动时页框是空的, 前几次缺页只是「装入」, 不触发置换。问「置换次数」时必须扣掉这部分。

六、页框分配·驻留集·工作集·抖动

策略组合 (2015-30): 「固定分配 + 全局置换」**不能组合使用**——固定分配意味着每个进程的页框数不变, 而全局置换允许从别的进程那里抢页框, 自相矛盾。可组合的三种: 可变分配 + 全局置换、可变分配 + 局部置换、固定分配 + 局部置换。

抖动 (thrashing) (2011-29): 刚被换出的页很快又要访问, 于是又换出别的页, 频繁置换, 大部分时间花在换页上。**有效措施: 撤销部分进程** (减少所需页面数)。无效: 增加对换区容量、提高用户进程优先级。

工作集 $w(k, t)$ (2016-29): 最近 k 次内存访问所访问过的页面集合, **重复的只算一次**。

最少页框数取决于 (2025-27): **指令系统的寻址方式**——支持间接寻址或跨页访问就需要更多页框。不是代码段长度, 不是虚拟/物理地址空间大小。

影响缺页率的 (2022-30): 置换算法 ✓、工作集大小 ✓、进程数量 ✓。**不影响: 页缓冲队列的长度 $\times$** ——它只影响页面置换的**速度**, 不影响缺页率。

影响有效 (平均) 访存时间的 (2020-28): 缺页率、磁盘读写时间、内存访问时间、执行缺页处理程序的 CPU 时间——**四项全选**。

能降低平均访存时间的 (2026-29): TLB ✓、工作集 ✓、页表缓冲队列 ✓。**多级页表 $\times$** ——它省的是空间, 反而增加地址转换步数, 延迟更大。

页框数与位图 (2023-25): 页框数 = 物理内存 $\div$ 页大小; 位图大小 = 页框数 $\div$ 8 字节。

七、分段与段页式

最大段长 (2009-27): 逻辑地址 = 段号 + 段内位移。地址 m 位、段号 n 位 $\Rightarrow$ 位移 $m - n$ 位 $\Rightarrow$ 最大段长 = 2^{m-n} 。(32 位地址、段号 8 位 $\Rightarrow$ 段内位移 24 位 $\Rightarrow$ 最大段长 2^{24} B)

越界异常 (2016-28): 段表项里存了段长。段内地址 $>$ 段长 $\Rightarrow$ 越界异常。注意区分三种异常: 段缺失、越权、越界。

共享段 (2019-28):

- 物理内存中只保存**一份**段的内容 ✓

- 共享段的段表项物理地址**唯一** ✓
- 所有进程都不再使用时才能回收 ✓
- 不同进程中的段号可以不同 ✗

八、概念题：反复出现的错误说法

这一节每条都是真题里被设成**错误选项**的表述，直接背。

虚拟存储的实现基础（2012-25）

- ✗ 只能基于连续分配技术 → ✓ 只能建立在**离散分配**基础上（请求分页／请求分段／请求段页式）
- ✗ 容量只受外存容量限制，或只受内存容量限制 → ✓ 由 **CPU 的寻址范围**决定

虚拟地址空间（2023-28）

- ✓ 每个进程都有自己独立的虚拟地址空间
- ✓ malloc() 返回的是虚拟地址
- ✓ 数据段和代码段可以有不同的访问权限
- ✗ 虚拟地址的大小由主存和硬盘的大小决定 → ✓ 由虚拟内存管理机制和操作系统定义

共享数据（2023-30）：两个进程共享 data，各自页号 p_1 、 p_2 **可以不相等**；但页框号 $f_1 = f_2$ **一定相等**（映射到同一物理页框）。

内存映射文件（2025-30）

- ✓ 可实现进程间通信（多个进程映射同一文件，共享同一份物理页面）
- ✓ 将文件映射到进程的**虚拟地址空间**
- ✗ 「实现了页面到磁盘块的映射」——方向说反了

逻辑地址形成于哪个阶段（2011-30）：**链接阶段**。编译产生的是各模块都从 0 开始、仅相对于本模块的地址；链接器完成重定位后，才形成整个程序的完整逻辑地址空间。

进程／线程切换时要更新什么（2018-45、2024-25、2025-23）

- 进程切换要更新：PC、栈基址寄存器、页表基址寄存器（PDBR）
- 同一进程内线程切换：PDBR 不更新（线程共享地址空间）
- 内核中断向量表基址寄存器：系统级，操作系统启动时设置一次，切换时不更新

九、虚拟存储 × Cache 交叉

解题依据：页大小 $= 2^k \implies$ 虚拟地址和物理地址的**低 k 位完全相同**。由此：

- 页内偏移那部分，可以直接用**虚拟地址的位**做 Cache 索引（VA 索引）
- 页号那部分，必须翻译成物理页号之后才能用（PA 索引）

2019-46 就是这条：页大小 4KB $\Rightarrow$ 低 12 位相同 $\Rightarrow$ call 指令虚拟地址的低 12 位即物理地址低 12 位 $\Rightarrow$ 取其中第 6~9 位得 Cache 组号。

两个层次的对照（2024-16、2016-45）

	Cache--主存	主存--外存
交换单位	主存块	页
替换算法	硬件实现	软件实现
写策略	可直写、可回写	总是回写
映射方式	直接/组相联/全相联	页式映射

主存--外存若采用直接映射，替换算法就完全失效了——2024-16 的 D 选项错误的正是这里。

缺失代价（2016-45）：缺页的开销远大于 Cache 缺失——缺页要访问磁盘，Cache 缺失只需访问主存（或说：缺页由软件处理，Cache 缺失由硬件处理）。

十、计算模板

反复用到的算式汇总：

1. 页内偏移位数 $k = \log_2(\text{页大小})$
2. 虚页号位数 = 虚拟地址位数 $-k$
3. 页框号位数 = 物理地址位数 $-k$
4. 物理地址 = 页框号 || 页内偏移（**拼接**）
5. 页表项数 = $2^{\text{虚页号位数}}$
6. 页表大小 = 页表项数 $\times$ 页表项大小
7. 一页可放页表项数 = 页大小 $\div$ 页表项大小
8. 页目录表项数 = 页表项总数 $\div$ 一页可放页表项数
9. 页表项物理地址 = 页表起始物理地址 + 页号 $\times$ 页表项大小
10. TLB 组数 = 表项数 $\div$ 路数；TLB 标记位数 = 虚页号位数 $-\log_2(\text{组数})$
11. 数组跨页数 = $\lceil (\text{起始页内偏移} + \text{数组字节数}) \div \text{页大小} \rceil$
12. 平均访问时间 = 命中时间 + 缺失率 $\times$ 缺失代价
13. 页框数 = 物理内存 $\div$ 页大小；位图大小 = 页框数 $\div 8$